

Tiny LLM Demo — Principi della rete neurale

Glossario, intuizione, e flusso completo dal training al prompt, con riferimenti al codice

Indice

Introduzione

Parte 1 — Glossario

Parte 2 — Come "ragiona" davvero una rete neurale

2.1 Non è un motore di regole

2.2 "Ragionare" = fare previsioni statistiche molto sofisticate

2.3 La conoscenza vive nei pesi, non in un database

2.4 Il training è ricerca, non programmazione

Parte 3 — Il flusso di training, passo per passo

Step 0 — Preparazione dei dati (prima del training vero e proprio)

Step 1 — Inizializzazione del modello

Step 2 — Il ciclo di training (ripetuto per ogni epoch)

Step 3 — Ripetizione

Step 4 — Salvataggio

Parte 4 — Il flusso operativo: dal prompt al risultato

Step 1 — Caricamento del modello allenato

Step 2 — Tokenizzazione dell'input

Step 3 — Forward pass (identico a quello del training, ma senza imparare nulla)

Step 4 — Da punteggi grezzi a probabilità

Step 5 — Scelta del prossimo token

Step 6 — Controllo di stop

Step 7 — Ripetizione autoregressiva

Step 8 — Composizione del risultato finale

Parte 5 — Training vs Inferenza: la differenza che conta davvero

Parte 6 — Applicato a un vero LLM: cosa cambia e cosa no

Riferimenti

Introduzione

Questo documento è il complemento concettuale della *Guida al codice*: lì abbiamo visto **come è scritto** il progetto, qui vediamo **perché funziona** — cosa succede realmente dentro la rete, come "impara", e come si comporta quando le dai un input.

Il documento segue quattro parti:

1. **Glossario** — tutti i termini tecnici che incontrerai, spiegati in una frase o due, prima di usarli.
2. **Come "ragiona" una rete neurale** — l'intuizione di fondo, senza formule.
3. **Il flusso di training** — passo per passo, con riferimento diretto al codice in `train.py`.
4. **Il flusso operativo (inferenza)** — dal prompt al risultato, con riferimento a `generate.py`.

Nessuna formula matematica: l'obiettivo è un modello mentale corretto e utilizzabile, non un corso di algebra lineare.

Parte 1 — Glossario

Termini elencati nell'ordine in cui li incontrerai leggendo il resto del documento, non in ordine alfabetico — così ogni definizione si appoggia sulla precedente.

Rete neurale — un sistema fatto di numeri organizzati in strati, che trasforma un input numerico in un output numerico. Non "sa" nulla a priori: il suo comportamento è determinato interamente dai valori di questi numeri, che vengono aggiustati durante il training.

Parametro (o peso) — uno dei numeri interni della rete che viene aggiustato durante il training. Nel nostro progetto, sono i numeri dentro `nn.Embedding` e `nn.Linear`: 255 in tutto. "Allenare un modello" significa, letteralmente, trovare buoni valori per questi numeri.

Tensore — la struttura dati con cui PyTorch rappresenta i numeri (input, parametri, output). Pensalo come un array multidimensionale ottimizzato per il calcolo. `torch.tensor([2, 5, 1])` è un tensore.

Token — l'unità minima di testo che il modello manipola. Nel nostro progetto un token è una parola intera (es. "gatto"); in un vero LLM è tipicamente un pezzo di parola.

Token speciale — un token che non corrisponde a nessuna parola del testo, ma a un *segnale*. Nel progetto ce n'è uno solo: `<end>` (`END` in `data.py`), che significa "la frase è finita". I modelli veri ne hanno diversi (fine sequenza, inizio/fine turno di conversazione, ecc.), tutti trattati dalla rete esattamente come le parole normali: sono id nel vocabolario.

Vocabolario — l'insieme di tutti i token che il modello conosce. Nel progetto ha 15 token: le 14 parole del corpus più `<end>` (`build_vocab()` in `data.py`); un LLM moderno ne ha tra 100.000 e 200.000.

Embedding — la rappresentazione numerica del significato di un token: un vettore (lista di numeri) che la rete impara durante il training. Token usati in contesti simili tendono ad avere embedding simili. Nel codice: `nn.Embedding` in `model.py`.

Forward pass — l'atto di far scorrere un input attraverso la rete per ottenere un output. Nel codice: la chiamata `model(x)`, che internamente esegue il metodo `forward()`.

Logit — il punteggio grezzo, non ancora normalizzato, che il modello assegna a ciascun token del vocabolario come "candidato successivo". Non è una probabilità (può essere negativo, non somma a 1).

Softmax — l'operazione che trasforma i logit in probabilità vere e proprie (numeri tra 0 e 1 che sommano a 100%). Nel codice: `torch.softmax(...)`.

Loss (funzione di perdita) — un singolo numero che misura quanto le predizioni del modello si discostano dalla risposta corretta. Loss alta = modello molto sbagliato. Nel codice: `nn.CrossEntropyLoss()`.

Gradiente — la "direzione e intensità" di correzione necessaria per ciascun parametro, per ridurre la loss. Calcolato automaticamente da PyTorch con `loss.backward()`.

Backpropagation — l'algoritmo che calcola i gradienti di tutti i parametri della rete, partendo dalla loss e "tornando indietro" strato per strato. È il meccanismo dietro `loss.backward()`.

Optimizer — il componente che, conoscendo i gradienti, applica effettivamente la correzione ai parametri. Nel codice: `torch.optim.Adam(...)`, invocato con `optimizer.step()`.

Learning rate — quanto è "grande" ogni passo di correzione applicato dall'optimizer. Troppo alto: il training diventa instabile. Troppo basso: impiega moltissimo tempo a migliorare. Nel codice: parametro `lr` di `train_model()`.

Epoch — un intero giro di training su tutto il dataset (forward → loss → backward → step). Il nostro progetto ne fa 200 di default (`epochs=200`).

Dataset di training — l'insieme di esempi (input, risposta corretta) su cui il modello si allena. Nel progetto: le 32 coppie parola-attuale/parola-successiva costruite da `build_dataset()`, comprese quelle in cui la "parola successiva" è `<end>`.

Inferenza — l'uso del modello già allenato per produrre un output su un nuovo input, senza più modificare i suoi parametri. Nel codice: le funzioni in `generate.py`.

Autoregressivo — un modo di generare testo in cui ogni nuovo token prodotto viene riutilizzato come input per generare il token successivo, uno alla volta. È il meccanismo del ciclo `for` dentro `generate()`.

Sampling (campionamento) e temperature — il modo in cui si sceglie il prossimo token dato un insieme di probabilità: non sempre il più probabile, ma uno scelto "a caso pesato". La temperature regola quanto questa scelta è prevedibile (bassa) o varia (alta). Nel progetto il default è 0.6 (`TEMPERATURE_DEFAULT` in `generate.py`).

Greedy decoding — il caso limite del sampling: niente casualità, si prende sempre il token con la probabilità più alta. Stesso input, stesso output, ogni volta. Nel progetto si attiva con `temperature=0`.

Criterio di stop — la regola che decide quando la generazione si ferma. Nel progetto sono due: un numero massimo di parole (`n parole`) oppure il modello che predice `<end>`, quello che arriva prima.

Checkpoint — il file su disco (`checkpoint.pt`) che contiene lo stato allenato del modello (tutti i suoi parametri) più il vocabolario, così da poterlo riusare senza ri-allenare da zero.

Overfitting (menzionato ma non presente nel nostro mini-progetto) — quando un modello "memorizza" i dati di training invece di generalizzare, e si comporta male su input mai visti. Con un dataset di 6 frasi, il nostro modello è quasi tutto memoria — è un limite di scala, non un errore di progettazione.

Parte 2 — Come "ragiona" davvero una rete neurale

2.1 Non è un motore di regole

Un sistema a regole scritto a mano (tipo `if/else` a catena) segue una logica esplicita decisa da un programmatore. Una rete neurale **non ha nessuna regola scritta**: ha solo numeri, e il suo comportamento emerge dal modo in cui quei numeri sono stati aggiustati osservando dati.

Non "capisce" che "gatto" e "cane" sono entrambi animali domestici nel senso in cui lo capiresti tu. Ha semplicemente imparato che, statisticamente, questi due token compaiono in contesti molto simili nel testo che ha visto (entrambi seguiti da "dorme", "gioca", "mangia") — e i loro embedding finiscono per essere vettori simili come *effetto collaterale* di questa statistica, non per una definizione esplicita di "animale".

2.2 "Ragionare" = fare previsioni statistiche molto sofisticate

Quando un LLM (o il nostro TinyLM) genera testo, sta rispondendo a un'unica domanda ripetuta:

| "dato quello che è venuto prima, qual è il token più plausibile adesso?"

Non c'è pianificazione, non c'è verifica di verità, non c'è comprensione nel senso umano. C'è un pattern-matching estremamente potente su regolarità statistiche del linguaggio — che a scala enorme (miliardi di parametri, migliaia di miliardi di parole viste) produce comportamenti che *sembrano* ragionamento, perché il linguaggio umano stesso incorpora logica, causalità e struttura, e il modello ne ha assorbito i pattern.

Il nostro progetto rende questo trasparente: con 6 frasi il modello "impara" solo statistiche banali (dopo "dorme" viene sempre "sul"; dopo "divano" viene sempre la fine della frase), ma il principio è identico a quello di un LLM da centinaia di miliardi di parametri.

Vale anche per il *sapersi fermare*: il modello non "decide" che la frase è completa. Ha semplicemente visto che, nel corpus, dopo "divano" e "tappeto" e "palla" c'è sempre `<end>`, e quindi lo predice. La fine della frase è un token come gli altri.

2.3 La conoscenza vive nei pesi, non in un database

Non c'è nessuna tabella "gatto → dorme sul divano" salvata da qualche parte in modo esplicito. Tutto ciò che il modello "sa" è codificato, in modo distribuito e non ispezionabile a occhio, nei valori numerici di `self.embedding` e `self.linear`. Se apri `checkpoint.pt` e guardi i numeri grezzi, non troverai nulla di leggibile — è *knowledge* compresso in forma puramente numerica, decodificabile solo facendolo passare di nuovo attraverso la rete.

2.4 Il training è ricerca, non programmazione

Allenare una rete non è scrivere istruzioni: è *cercare*, per tentativi guidati (gradiente dopo gradiente), la combinazione di numeri che minimizza gli errori su un insieme di esempi. È più vicino a "far evolvere una soluzione" che a "scriverla".

Un corollario pratico, che nella demo si vede bene: se cambia il corpus (aggiungi una frase), non si "aggiorna" il modello aggiungendo una regola — si ributta via tutto e si ri-cerca da zero. Con 255 parametri ci vuole meno di un secondo; con miliardi di parametri ci vogliono mesi e datacenter interi, ed è per questo che i modelli veri hanno una "data di taglio" della conoscenza.

Parte 3 — Il flusso di training, passo per passo

Riferimento al codice: `train_model()` in `src/train.py`.

Step 0 — Preparazione dei dati (prima del training vero e proprio)

```
stoi, itos = build_vocab(corpus)      # src/data.py
X, Y = build_dataset(corpus, stoi)    # src/data.py
```

Il testo viene trasformato in coppie numeriche (X = parola attuale, Y = parola corretta successiva). Ogni frase viene prima chiusa con il token `<end>`, che diventa la risposta corretta della sua ultima parola:

```
"il gatto dorme sul divano" -> (il → gatto), (gatto → dorme), (dorme → sul),
                               (sul → divano), (divano → <end>)
```

Questo accade una sola volta, prima che inizi il ciclo di training. Senza il token di chiusura, parole come "divano" (che compaiono solo a fine frase) non sarebbero mai *input* di nessun esempio, e il modello non avrebbe idea di cosa fare dopo di loro.

Step 1 — Inizializzazione del modello

```
model = TinyLM(vocab_size=len(stoi), embedding_dim=embedding_dim)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
```

Il modello parte con pesi **casuali**: a questo punto, non sa assolutamente nulla. `criterion` e `optimizer` sono gli strumenti che useremo per correggerlo, non fanno parte del modello stesso.

Step 2 — Il ciclo di training (ripetuto per ogni epoch)

```
for epoch in range(epochs):
    optimizer.zero_grad()          # (a)
    logits = model(X)              # (b)
    loss = criterion(logits, Y)     # (c)
    loss.backward()                 # (d)
    optimizer.step()                # (e)
```

(a) Azzeramento dei gradienti. Puliamo i calcoli del giro precedente, per non sommarli a quelli nuovi.

(b) Forward pass. Ogni parola di `X` viene trasformata: prima in un embedding (il suo vettore di significato attuale), poi in un punteggio per ogni token del vocabolario (i logit). In questo momento il modello sta *tentando una previsione*, con i pesi che ha in quel momento.

(c) Calcolo della loss. Confrontiamo i logit prodotti con le risposte corrette (`Y`). Il risultato è un numero: quanto, in media, il modello si è sbagliato su tutto il dataset in questo giro.

(d) Backpropagation. PyTorch calcola, per ciascun peso della rete, "di quanto e in che direzione" contribuisce all'errore appena misurato. Questo è il passo concettualmente più delicato: non stiamo ancora correggendo nulla, stiamo solo *misurando la responsabilità* di ogni singolo numero nell'errore totale.

(e) Aggiornamento dei pesi. L'optimizer applica finalmente le correzioni calcolate al passo (d), un piccolo passo alla volta (regolato da `lr`).

Step 3 — Ripetizione

```
storico_loss.append(loss.item())
```

Questo ciclo si ripete per `epochs` volte (200 di default). Ogni ripetizione, la loss dovrebbe scendere un po': il modello sta *convergendo* verso pesi che spiegano meglio il nostro corpus.

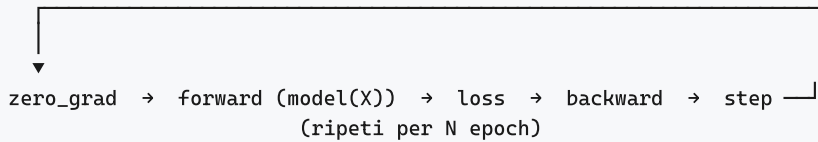
Tracciamo la loss ad ogni epoch solo per poterla osservare (è quello che stampiamo ogni 50 epoch, ed è quello che la demo mostra scendere in tempo reale). Nota che la loss non arriva mai a zero, ed è giusto così: dopo "il" il corpus contiene sia "gatto" che "cane" in ugual misura, quindi la previsione migliore possibile è "50 e 50", non una certezza.

Step 4 — Salvataggio

```
save_checkpoint(model, stoi, itos)
```

A training concluso, i pesi finali (il "sapere" del modello) vengono serializzati su disco insieme al vocabolario. Da questo momento, il modello è pronto per l'inferenza — non verrà più modificato finché non lo riallenerai di nuovo.

Schema riassuntivo del ciclo:



Parte 4 — Il flusso operativo: dal prompt al risultato

Riferimento al codice: `generate()` e `genera_da_frase()` in `src/generate.py`.

Questa è la sequenza che accade ogni volta che lanci `python -m src.generate <parola>` o scegli "Genera testo" nella demo.

Step 1 — Caricamento del modello allenato

```
modello, stoi, itos = load_checkpoint()
```

I pesi salvati durante il training vengono ricaricati in un `TinyLM` nuovo. Da qui in poi, i pesi **non cambiano più**: siamo in modalità inferenza, non training. (Nella demo interattiva questo passo non c'è: il modello appena allenato è già in memoria.)

Step 2 — Tokenizzazione dell'input

```
if parola_iniziale not in stoi:
    raise ValueError(f'"{parola_iniziale}" non è nel vocabolario. ...')

x = torch.tensor([stoi[parola]])
```

La parola che hai passato viene convertita nel suo id numerico, usando lo stesso identico vocabolario (`stoi`) costruito durante il training. Se la parola non esiste nel vocabolario, il modello non ha *nessun modo* di rappresentarla: non c'è una riga di embedding per lei. Per questo scatta subito un `ValueError` con l'elenco delle parole valide, invece di un crash più avanti. (Un LLM vero aggira il problema spezzando le parole sconosciute in pezzi più piccoli che conosce; noi no.)

Step 3 — Forward pass (identico a quello del training, ma senza imparare nulla)

```
logits = modello(x)
```

L'input attraversa `nn.Embedding` poi `nn.Linear`, esattamente come durante il training. La differenza è che qui siamo dentro `with torch.no_grad():` — non ci serve calcolare i gradienti, perché non correggeremo nessun peso.

Step 4 — Da punteggi grezzi a probabilità

```
probabilita = torch.softmax(logits / temperature, dim=-1)
```

I logit vengono convertiti in probabilità: 15 numeri (uno per token, compreso `<end>`) che sommano a 1. `temperature` regola quanto questa distribuzione è "decisa" (bassa) o "aperta a sorprese" (alta). Il default 0.6 è volutamente prudente: con un corpus così piccolo, a 1.0 il modello pescherebbe troppo spesso continuazioni rare.

Step 5 — Scelta del prossimo token

```
if temperature <= 0:
    prossimo_id = int(torch.argmax(logits, dim=-1).item()) # greedy
else:
    prossimo_id = int(torch.multinomial(probabilita, num_samples=1).item()) # sampling
parola = itos[prossimo_id]
```

Nel caso normale (**sampling**) viene scelto un token *pesato* dalle probabilità appena calcolate — non necessariamente il più probabile in assoluto. È il motivo per cui la stessa parola di partenza può dare frasi diverse a ogni run.

Con `temperature=0` (**greedy**) si salta il softmax e si prende direttamente il logit più alto: nessuna casualità, output riproducibile. È quello che usano i test automatici per poter fare asserzioni esatte.

In entrambi i casi l'id numerico viene poi ri-tradotto in parola leggibile tramite `itos`.

Step 6 — Controllo di stop

```
if parola == END:
    break
```

Se il token scelto è `<end>`, la generazione si ferma qui: il modello ha "detto" che la frase è finita. Il token non viene aggiunto all'output — è un segnale interno, non una parola. Se invece non è `<end>`, la parola viene accodata alla frase e si prosegue.

L'altro criterio di stop è `n_parole`: se il modello non produce mai `<end>`, ci fermiamo comunque dopo quel numero massimo di parole. È la stessa coppia di regole di un LLM vero (token di fine sequenza + limite massimo di token in output).

Step 7 — Ripetizione autoregressiva

```
for _ in range(n_parole):
    ...
    parola = itos[prossimo_id] # diventa l'input del giro successivo
```

La parola appena generata diventa l'input del passo successivo (si torna allo Step 3). È qui che il nostro modello, guardando solo una parola alla volta, costruisce comunque una sequenza intera — ed è anche qui che si vede il suo limite principale (nessuna memoria oltre l'ultima parola).

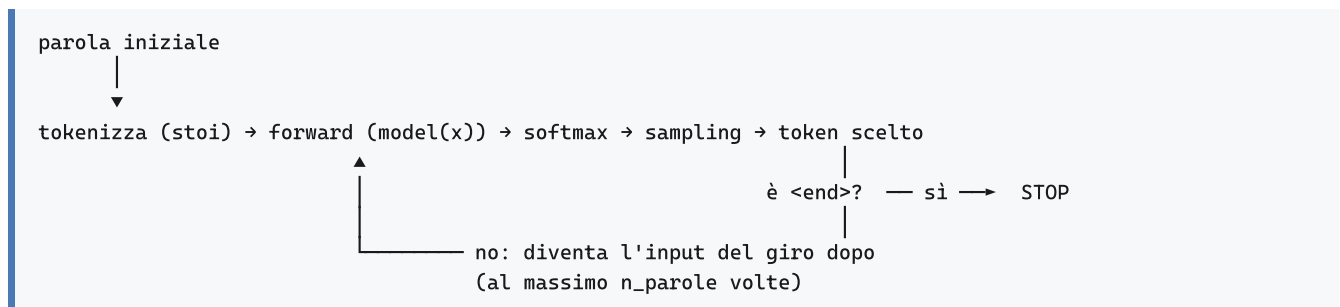
Step 8 — Composizione del risultato finale

```
return " ".join(frase)
```

Tutti i token generati (compreso quello iniziale, escluso `<end>`) vengono uniti in una stringa leggibile: il risultato che vedi stampato a terminale.

Nel caso di `genera_da_frase()` (completamento di una frase intera), c'è uno *step* o aggiuntivo: solo **l'ultima parola** della frase in input viene effettivamente data in pasto al modello (Step 2 in poi); il resto della frase viene semplicemente riattaccato davanti al risultato, senza che il modello lo abbia mai "visto" — per questo, come discusso, non è vera comprensione del contesto lungo.

Schema riassuntivo del ciclo di inferenza:



Parte 5 — Training vs Inferenza: la differenza che conta davvero

	Training	Inferenza
Obiettivo	Trovare buoni pesi	Usare i pesi già trovati
I pesi cambiano?	Sì, ad ogni epoch	Mai
Serve la risposta corretta (Y)?	Sì, per calcolare la loss	No, non esiste una "risposta corretta" da verificare
Gradiente calcolato?	Sì (<code>loss.backward()</code>)	No (<code>torch.no_grad()</code>)
Casualità	Solo nei pesi iniziali	Nel sampling (a meno di greedy)
Nel codice	<code>train_model()</code> in <code>train.py</code>	<code>generate()</code> in <code>generate.py</code>
Quante volte gira	Ripetuto per molti epoch, su tutto il dataset insieme	Un forward pass per ogni token generato, finché non esce <code><end></code> o si raggiunge <code>n_parole</code>

Questo è probabilmente **il concetto singolo più importante** da avere chiaro: un LLM "impara" una sola volta (fase di training, costosa, offline), poi viene usato moltissime volte in inferenza (economica, in tempo reale) senza mai più modificare i suoi pesi — a meno di un successivo, nuovo, esplicito ciclo di training (o fine-tuning). Quando chatti con un assistente AI, stai facendo *solo* inferenza: la conversazione non gli "insegna" nulla in modo permanente.

Parte 6 — Applicato a un vero LLM: cosa cambia e cosa no

- **Stessa identica logica di fondo:** tokenizzazione → embedding → forward pass → softmax → sampling → stop su token di fine sequenza, e in training: loss → backward → optimizer step. Nessun principio nuovo.
- **Contesto:** il nostro modello guarda 1 sola parola precedente. Un vero LLM usa l'**attention**, un meccanismo (non trattato in questo progetto) che permette al forward pass di guardare contemporaneamente migliaia o milioni di token precedenti, pesando quali sono più rilevanti per la

previsione corrente. È esattamente il limite che `genera_da_frase()` rende visibile: `il cane gioca` può diventare `il cane gioca con la carne` perché il modello vede solo "gioca".

- **Tokenizzazione:** noi usiamo parole intere e una parola sconosciuta è un errore. Un LLM usa *sub-word tokens* (pezzi di parola): qualunque testo, in qualunque lingua, si può sempre spezzare in token noti, quindi non esiste "parola fuori vocabolario".
- **Temperature e sampling:** il parametro `temperature` che trovi nelle API dei modelli commerciali è letteralmente lo stesso numero del nostro `generate()`, con lo stesso effetto. In più i modelli veri usano tecniche di campionamento più raffinate (top-k, top-p) per tagliare la coda delle scelte improbabili.
- **Scala:** vocabolario, numero di parametri, quantità di dati di training sono ordini di grandezza più grandi, ma l'unità di misura concettuale — parametro, embedding, loss, gradiente, epoch — è esattamente la stessa. Il nostro training fa 200 passate sullo stesso corpus; un LLM vede ogni pezzo di testo tipicamente una o poche volte, perché di testo ne ha a disposizione molto più di quanto riesca a leggere.
- **Fasi aggiuntive nei LLM moderni** (non presenti nel nostro progetto): dopo il training di base (*pre-training*) su enormi quantità di testo, i modelli conversazionali passano da ulteriori fasi di *fine-tuning* e allineamento tramite feedback umano (RLHF e tecniche derivate), per renderli più utili e sicuri da usare in conversazione. Il nostro progetto copre solo l'equivalente del pre-training di base.

Riferimenti

- Documentazione ufficiale PyTorch — Autograd: <https://pytorch.org/docs/stable/notes/autograd.html>
- Documentazione ufficiale PyTorch — `nn.Module`: <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>
- Andrej Karpathy, "Let's build GPT: from scratch, in code, spelled out" — video di riferimento per l'approccio didattico bottom-up
- Jay Alammar, "The Illustrated Transformer" — per approfondire l'attention, il pezzo mancante rispetto al nostro mini-modello